

Solving Recurrence Equation

- The recurrence relation can be solve by following methods
- Substitution Method
- Tree Method
- Master Method

Substitution method

- It is a kind of method in which a guess for the solution is made.
- There are two type of substitution
 - Forward substitution
 - Backward substitution
- **Forward Substitution** : It make use of an initial condition in the initial term and value for the next term is generated.
- Process is continued until some formula is guessed.
- **Backward Substitution** : In this backward values are substituted recursively in order to derive some formula.

Forward Substitution method - Example

- Consider a recurrence relation $T(n) = T(n - 1) + n, T(0) = 0$.

$$T(0) = 0$$

↳ ①

$$\begin{aligned} n=1 \quad T(1) &= T(1-1) + 1 \\ &= T(0) + 1 \\ &= 0 + 1 \end{aligned}$$

$$\begin{aligned} n=2 \quad T(2) &= T(2-1) + 2 \\ &= T(1) + 2 \\ &= 0 + 1 + 2 \end{aligned}$$

$$\begin{aligned} n=3 \quad T(3) &= T(2) + 3 \\ &= 0 + 1 + 2 + 3 \end{aligned}$$

$$T(4) = 0 + 1 + 2 + 3 + 4$$

⋮

$$T(n) = 0 + 1 + 2 + 3 + \dots + n$$

$$= \frac{n(n+1)}{2}$$

$$\in \Theta(n^2)$$

Backward Substitution method - Example

- Consider a recurrence relation $T(n) = T(n-1) + n$, $T(0) = 0$.

$$T(n) = T(n-1) + n \quad \text{--- (1)}$$

$$n = n-1$$

$$T(n-1) = T(n-1-1) + (n-1) \\ = T(n-2) + (n-1) \quad \text{--- (2)}$$

Use (2) in (1) $T(n) = T(n-2) + (n-1) + n \quad \text{--- (3)}$

$$n = n-2$$

$$T(n-2) = T(n-3) + (n-2) \quad \text{--- (4)}$$

Use (4) in (3)

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

⋮

$$T(n) = T(0) + 1 + \dots + (n-2) + (n-1) + n$$

Using $n=1$
Using $n=0$

$$T(n) = 0 + 1 + 2 + 3 + \dots + (n-2) + (n-1) + n$$

$$T(n) = \frac{n(n+1)}{2} \\ = \Theta(n^2) //$$

Solve the recurrence relation $T(n) = T(n - 1) + 1, T(0) = 0$.

$$n = n - 1 \quad T(n - 1) = T(n - 2) + 1$$

$$n = n - 2 \quad T(n - 2) = T(n - 3) + 1$$

$$T(n) = T(n - 1) + 1$$

$$= T(n - 2) + 1 + 1$$

$$= T(n - 3) + 1 + 1 + 1$$

⋮

$$= T(0) + 1 + 1 + \dots + n$$

$$= n$$

$$\in \Theta(n) //$$

∴

Solve the recurrence relation $T(n) = 2T(\frac{n}{2}) + n, T(1) = 1$.

$$\begin{aligned}
 n = n/2 \quad T(n/2) &= 2T(n/4) + n/2 \\
 n = n/4 \quad T(n/4) &= 2T(n/8) + n/4 \\
 T(n) &= 2T(n/2) + n \\
 &= 2 \cdot (2T(n/4) + n/2) + n = 4T(n/4) + 2n = 2^2 T(n/2^2) + 2n \\
 &= 4(2T(n/8) + n/4) + 2n = 8T(n/8) + 3n \\
 &= 2^3 T(n/2^3) + 3n
 \end{aligned}$$

$$\begin{aligned}
 &\vdots \\
 &= 2^k T(n/2^k) + k \cdot n \longrightarrow \textcircled{1} \\
 T(1) &= 1 \quad n/2^k = 1 \quad n = 2^k \quad k = \log n \\
 \text{Sub } n &= 2^k \\
 &= 2^{\log n} T(1) + \log n \cdot n \\
 &= \underline{2^{\log n}} + n \cdot \log n = \underline{\underline{\Theta(n \cdot \log n)}}
 \end{aligned}$$

Solve the recurrence relation $T(n) = T(\frac{n}{3}) + c, T(1) = 1$.

$$T(n) = T(n/3) + c$$

$$= T(n/9) + c + c = T(n/3^2) + 2c$$

$$= T(n/27) + c + c + c = T(n/3^3) + 3c$$

$\vdots$

$$= T(\underline{n/3^k}) + k \cdot c$$

$$T(1) = 1 \quad n/3^k = 1 \Rightarrow n = 3^k \Rightarrow k = \log_3 n$$

$$= T(1) + \log_3 n \cdot c$$

$$= 1 + c \log_3 n$$

$$\in \Theta(\log_3 n) //$$

Solve the recurrence relation $T(n) = T(n-1) + n^4, T(0) = 0$.

$$T(n) = \underline{T(n-1)} + n^4$$

$$= \underline{T(n-2)} + (n-1)^4 + n^4$$

$$= \underline{T(n-3)} + (n-2)^4 + (n-1)^4 + n^4$$

$$= \underline{T(n-4)} + (n-3)^4 + (n-2)^4 + (n-1)^4 + n^4$$

⋮

$$= \underline{T(n-k)} + (n-k+1)^4 + (n-k+2)^4 + \dots + n^4$$

$$\begin{aligned} T(0) &= 0 \\ T(n-k) &= 0 \\ n-k &= 0 \\ n &= k \end{aligned}$$

$$= \underline{T(0)} + 1^4 + 2^4 + \dots + n^4$$

$$= 0 + \sum_{k=1}^n k^4 = n^5$$

$$n + 2n + 3n + 4n$$

$$n(1 + 2 + 3 + \dots)$$

$$\sum_{k=1}^n k$$

$$\frac{n(n+1)}{2}$$

$$\text{Sub } k=n$$

$$\sum n^4 = \frac{(6n^3 + 9n^2 + n - 1)}{30}$$

Tree Method

- In this method, a recursion tree is built in which each node represents the cost of a single subproblem in the form of recursive function invocations.
- We sum up the cost at each level to determine the overall cost.
- This help us to make a good guess of time complexity.

Tree Method - Example

- Consider a recurrence relation $T(n) = 2T(\frac{n}{2}) + n, T(1) = 1$.

Recurrence

Nodes
 2^p

Tree

Cost

$$T(n)_{k=1}$$

$$2^1$$

$$T(n/2)_{k=2}$$

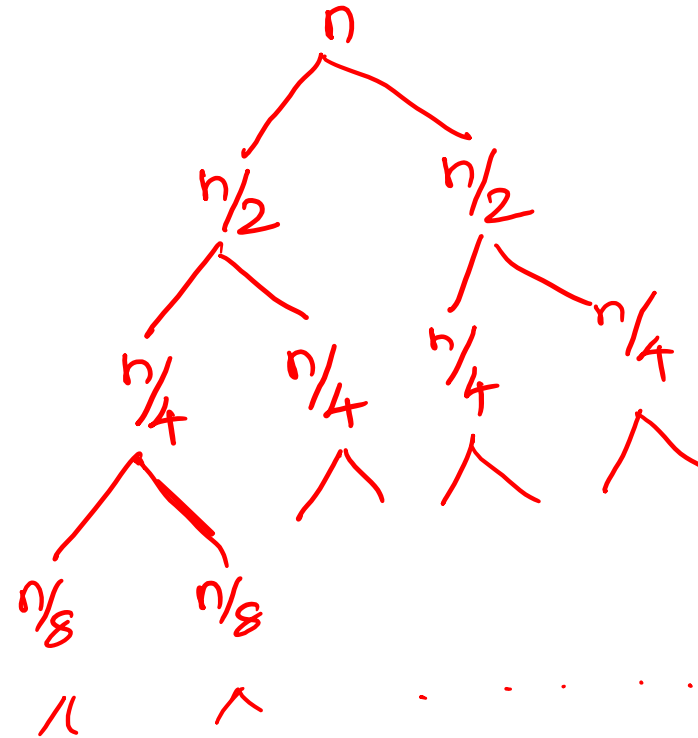
$$2^2$$

$$T(n/2^3)_{k=3}$$

$$2^3$$

$$T(n/2^k)$$

$$2^k$$



$$n$$

Initial is $T(1)$

$$n$$

$$T(n/2^k) = 1$$

$$n$$

$$n = 2^k$$

$$n$$

$$k = \log n$$

$$n$$

$$\sum_{i=0}^k n = n \sum_{i=0}^k 1$$

$$= n(k+1) = n \log n + n \in \Theta(n \log n)$$

Solve the recurrence relation $T(n) = 2T(\frac{n}{2}) + \textcircled{4n} T(1) = 1$.

Recursion

Nodes

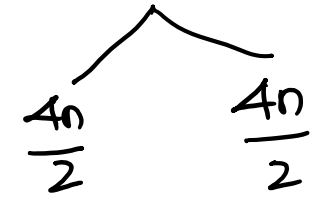
Tree
 $4n$

Cost
 $4n$

$T(1) = T(n/2^k)$

$T(n)$

2

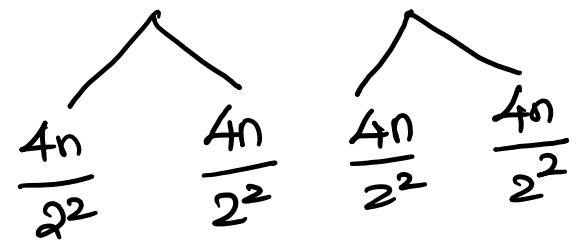


$4n$

$1 = n/2^k$

$T(n/2)$

2^2



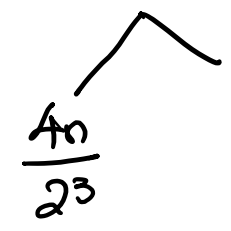
$4n$

$2^k = n$

$k = \log n$

$T(n/2^2)$

2^3



$\vdots$

$T(n/2^k)$

2^k



$$\sum_{i=0}^k 4n = 4n \sum_{i=0}^k 1$$

$$= 4n(1+k) = 4n + 4n \cdot \log n$$

$$\in \Theta(n \log n) //$$

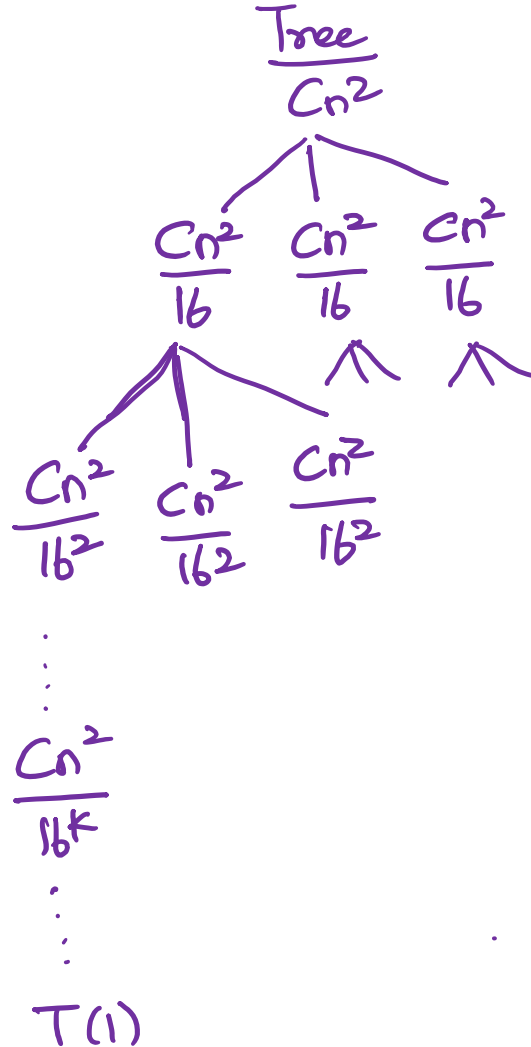
Solve the recurrence relation $T(n) = 3T(\frac{n}{4}) + cn^2, T(1) = 1$.

$$T(n) \quad 1 \quad k=0$$

$$T(n) \quad 3 \quad k=1$$

$$T(\frac{n}{16}) \quad 3^2 \quad k=2$$

$$T(\frac{n}{16^k}) \quad 3^k$$



$$\frac{Cost}{cn^2}$$

$$\frac{3 \cdot cn^2}{16}$$

$$\frac{9 \cdot cn^2}{16^2}$$

$$\frac{3^k \cdot cn^2}{16^k}$$

$$\left(\frac{3}{16}\right)^1 \cdot cn^2$$

$$\frac{3^2}{16^2} \cdot cn^2$$

$$\sum_{i=0}^k \left(\frac{3}{16}\right)^i \cdot cn^2 = cn^2 \sum_{i=0}^k \left(\frac{3}{16}\right)^i \in \Theta(n^2) //$$

$$1 + a + a^2 + a^3 + \dots$$

$$= \frac{a^{k+1} - 1}{a - 1}$$

Time Efficiency of Non-recursive Algorithm

- $\sum_{i=l}^u 1 = 1 + 1 + \cdots + 1 = (u - l) + 1.$
- $\sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{n^2}{2} \in \Omega(n^2).$
- $\sum_{i=1}^n i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3} \in \Omega(n^3).$
- $\sum_{i=1}^n a^i = 1 + a + a^2 + \cdots + a^n = \frac{a^{n+1}-1}{a-1}, \forall a \neq 1.$

Master's Method

$$\underline{n^{\log_b a}} \approx f(n)$$

- In this method, we can solve recurrence relation using a formula denoted by Master's method.
- $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, where $n \geq d$ and d is some constant.
- Then Master's theorem can be stated for efficiency analysis as
 1. If $f(n)$ is $O(n^{\log_b a - \epsilon})$ then $T(n) = \Theta(n^{\log_b a})$
 2. If $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$ then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
 3. If $f(n)$ is $\Omega(n^{\log_b a + \epsilon})$ then $T(n) = \Theta(f(n))$

Solve the recurrence relation $T(n) = 4T(\frac{n}{2}) + n$.

$$a=4 \quad b=2 \quad f(n)=n$$

$$n^{\log_b a} = n^{\log_2 4} = n^2$$

$f(n) \in O(n^{\log_2 4 - 1})$ by using Rule 1 in Master theorem

$$T(n) \in \Theta(n^{\log_b a})$$

$$\in O(n^2)$$

Solve the recurrence relation $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$.

$$f(n) = n \log n$$

$$a=2 \quad b=2$$

$$n^{\log_b a} = n^{\log_2 2} = n = n^{\log_2 2} \log^0 n \quad \text{where } k=0$$

Rule 2 in Master theorem

$$F(n) \text{ is } \Theta(n^{\log_b a} \log^k n)$$

$$\therefore F(n) \in \Theta(n^{\log_b a} \log^{k+1} n)$$

$$\in \Theta(n \log n) //$$

Solve the recurrence relation $T(n) = 8T\left(\frac{n}{2}\right) + n^2$.

Solve the recurrence relation $T(n) = T\left(\frac{n}{2}\right) + 1$.

$$n^{\log_2 1} = n^0 = 1$$

Use Rule 2: $f(n) \in \Omega(n^{\log_2 1} \log^0 n)$ $\therefore T(n) \in \Theta(n^{\log_2 1} \log^{k+1} n)$
 $\in \Theta(\log n) //$

Solve the recurrence relation $T(n) = k \cdot T\left(\frac{n}{k}\right) + n^2$, $T(1) = 1$ and k is any constant

Exercise

- $x(n) = x(n - 1) + 5$ for $n > 1, x(1) = 0$
- $x(n) = 3x(n - 1)$ for $n > 1, x(1) = 4$
- $X(n) = X(\frac{n}{2}) + n$ for $n > 1, x(1) = 1$
- $X(n) = x(\frac{n}{3}) + 1$ for $n > 1, x(1) = 1$

Thank u

Reference:

https://www.academia.edu/36827773/1_Asymptotic_Notation_pptx

<https://www.coursehero.com/file/54424142/03-Asymptotic-Notations-I-ppt/>

<https://www.authorstream.com/Presentation/dineshkavitha-431699-asymptotic-notation-1-education-ppt-powerpoint/>

https://www.academia.edu/36827773/1_Asymptotic_Notation_pptx

Books:

Mark A. Weiss, Data Structures & Algorithm Analysis in C++, 4th Edition, 2013, Pearson Education.

Alfred V. Aho, Jeffrey D. Ullman and John E. Hopcroft, Data Structures and Algorithms, 1983, Pearson Education.

Horowitz, Sahni and S. Anderson-Freed, Fundamentals of Data Structures in C, 2008, 2nd Edition, Universities Press.